ME381 ROBOTICS

Experiment No.: 7

Experiment Title: Kinematics of a five-bar parallel planar manipulator

Group No.: B6

Name: Pritam Priyadarshi

Roll no.: 230793

Results for the group task:

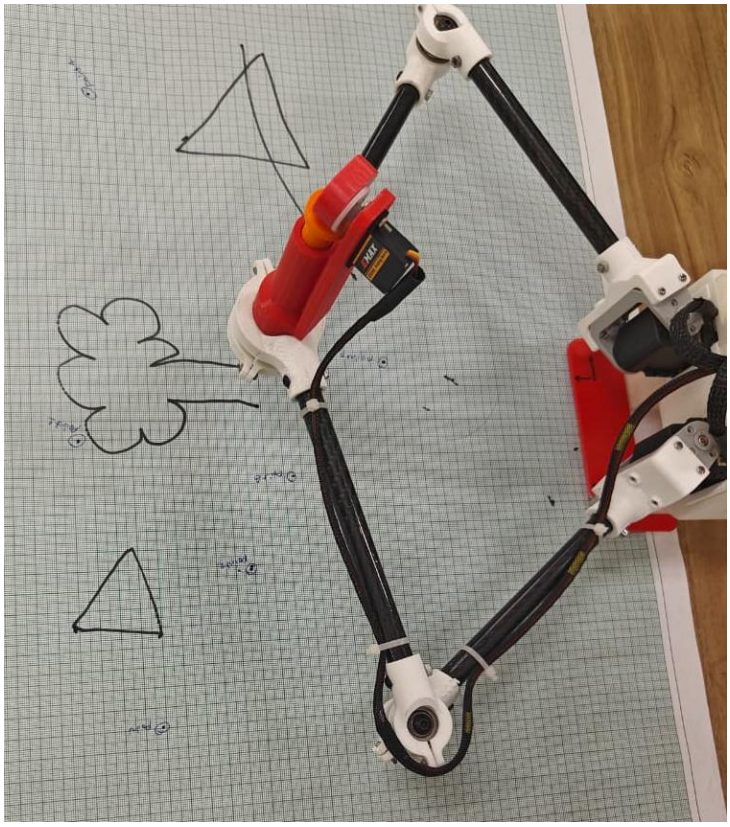
1. Forward Kinematics

Sr. no.	Left motor (radians)	Right motor (radians)	Calculated coordinates (X, Y) (MuJoCo) (units: mm)	Measured coordinates (X, Y) (units: mm)
1	0	0	(397, 0)	(397, 2)
2	1.57	-1.57	(108, 2)	(115, 55)
3	1.23	-0.451	(207, 78)	(217, 82)
4	1.23	0.00392	(256, 157)	(266, 159)
5	-0.29	-1.1	(290, -198)	(290, -197)
6	0.7374	-1.3	(109, -31)	(130, -33)

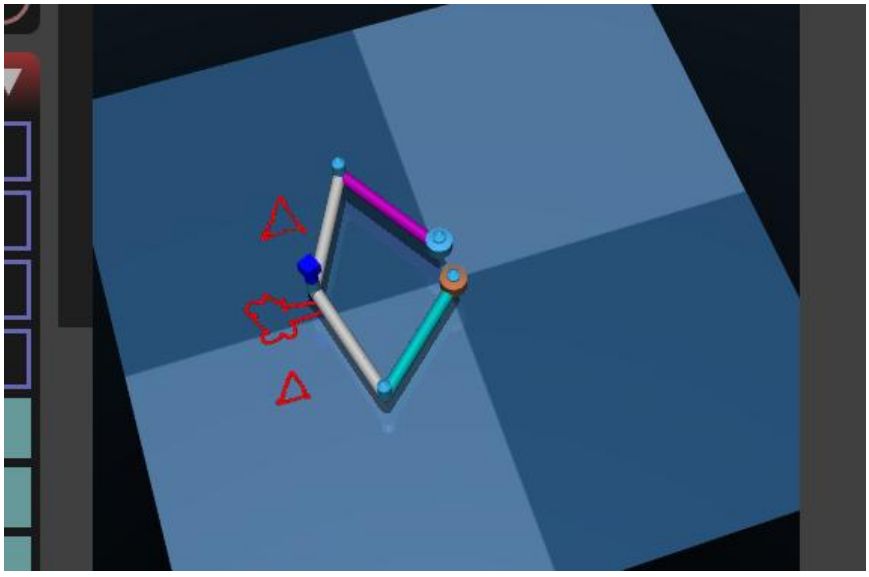
ME381 ROBOTICS

2. Visualising robot motion

Drawn Shape image:



Snapshot of the GUI:

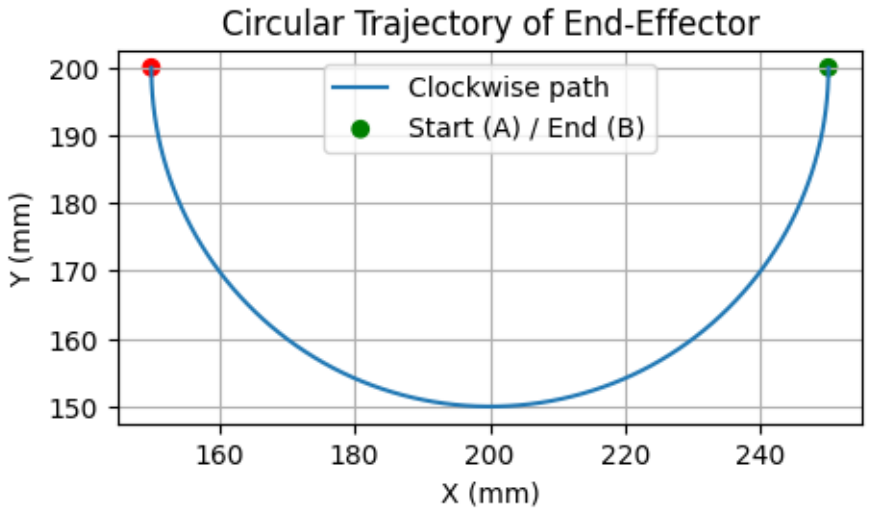


ME381 ROBOTICS

3. Trajectory planning of the robot end-effector

1.

```python
import numpy as np
import matplotlib.pyplot as plt

### Required Function ###
def circular_traj(center, radius, theta):
    cx, cy = center
    X = cx + radius * np.cos(theta)
    Y = cy - radius * np.sin(theta)   # minus for clockwise motion
    return X, Y
### Required Plot ###
O = (200, 200)
r = 50

# Generate theta values (A → B along lower semicircle)
theta = np.linspace(0, np.pi, 200)

# Compute circular coordinates
X, Y = circular_traj(O, r, theta)

# Plot the circular arc
plt.figure(figsize=(5,5))
plt.plot(X, Y, label='Clockwise path')
plt.scatter([X[0], X[-1]], [Y[0], Y[-1]], c=['g','r'], label='Start (A) / End (B)')
plt.gca().set_aspect('equal', adjustable='box')
plt.title('Circular Trajectory of End-Effector')
plt.xlabel('X (mm)')
plt.ylabel('Y (mm)')
plt.legend()
plt.grid(True)
plt.show()
```

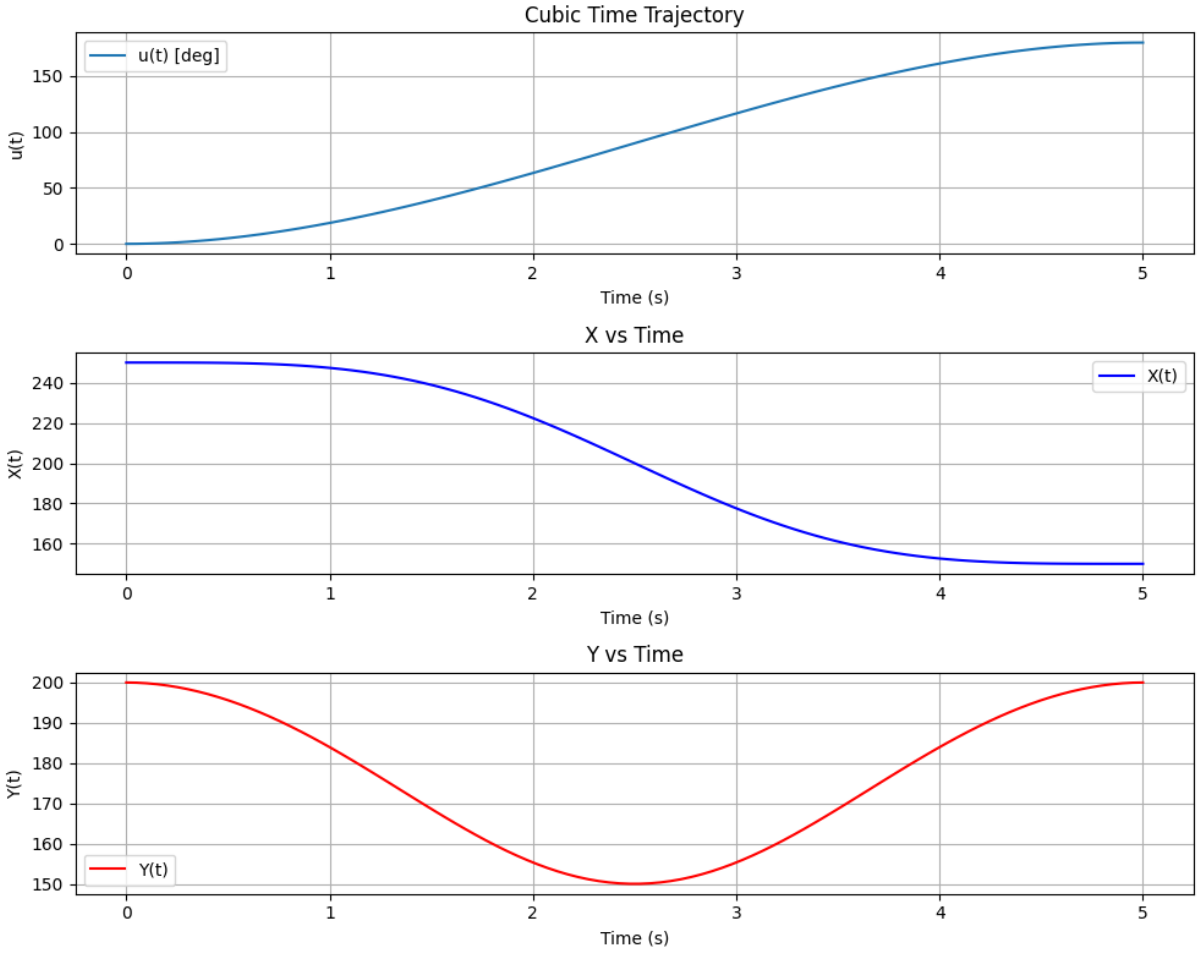


ME381 ROBOTICS

2.

```
### Required Plot ###
import numpy as np
import matplotlib.pyplot as plt

# Defining boundary conditions
t1 = 0.0        # start time
t2 = 5.0        # end time
u1 = 0.0        # initial position
u2 = 180.0       # final position
du1 = 0.0       # initial velocity
du2 = 0.0       # final velocity

# Sampling time
t_vals = np.linspace(t1, t2, 100)

# u(t)
u_vals = [cubic_time_traj(t, t1, t2, u1, u2, du1, du2) for t in t_vals]

# X(t) and Y(t)
u_rad = np.radians(u_vals)
X_vals = 200 + 50 * np.cos(u_rad)
Y_vals = 200 - 50 * np.sin(u_rad)

# Plot u(t), X(t), and Y(t)
plt.figure(figsize=(10, 8))

plt.subplot(3,1,1)
plt.plot(t_vals, u_vals, label="u(t) [deg]")
plt.xlabel("Time (s)")
plt.ylabel("u(t)")
plt.title("Cubic Time Trajectory")
plt.grid(True)
plt.legend()

plt.subplot(3,1,2)
plt.plot(t_vals, X_vals, 'b', label="X(t)")
plt.xlabel("Time (s)")
plt.ylabel("X(t)")
plt.title("X vs Time")
plt.grid(True)
plt.legend()

plt.subplot(3,1,3)
plt.plot(t_vals, Y_vals, 'r', label="Y(t)")
plt.xlabel("Time (s)")
plt.ylabel("Y(t)")
plt.title("Y vs Time")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```

ME381 ROBOTICS



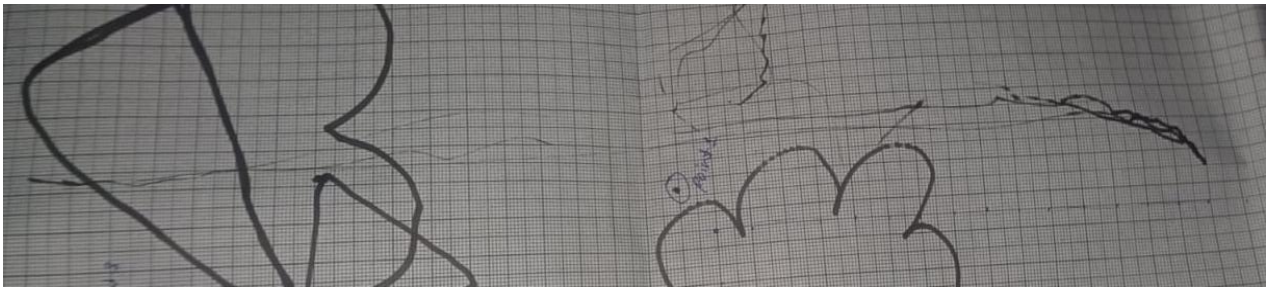
ME381 ROBOTICS

4. Trajectory tracking: straight-line segment

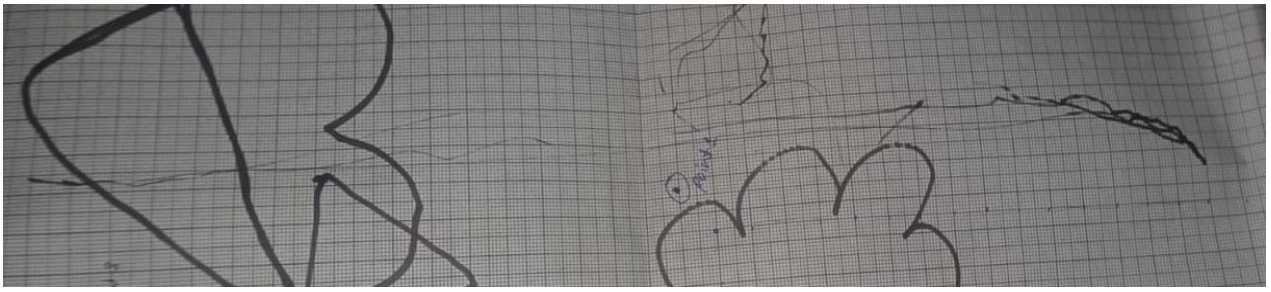
Table 1 Details of the straight-line segment

Sr no.	P1	P2	Duration (s)
1.	[0.3, 0.18]	[0.3, -0.15]	3
2.	[0.3, 0.18]	[0.3, -0.15]	1

1.

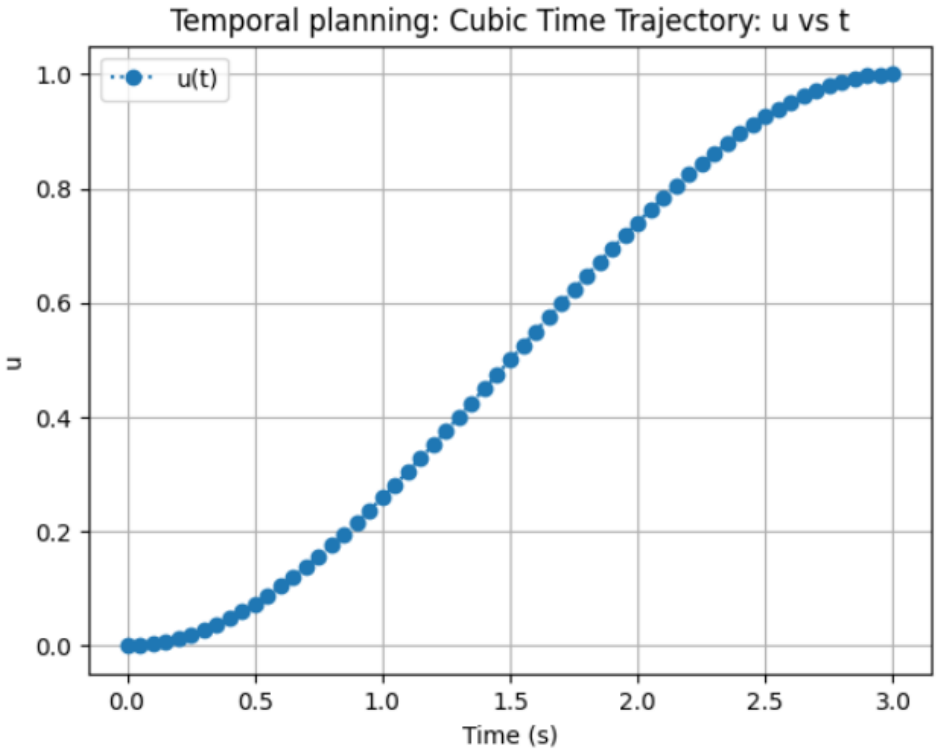


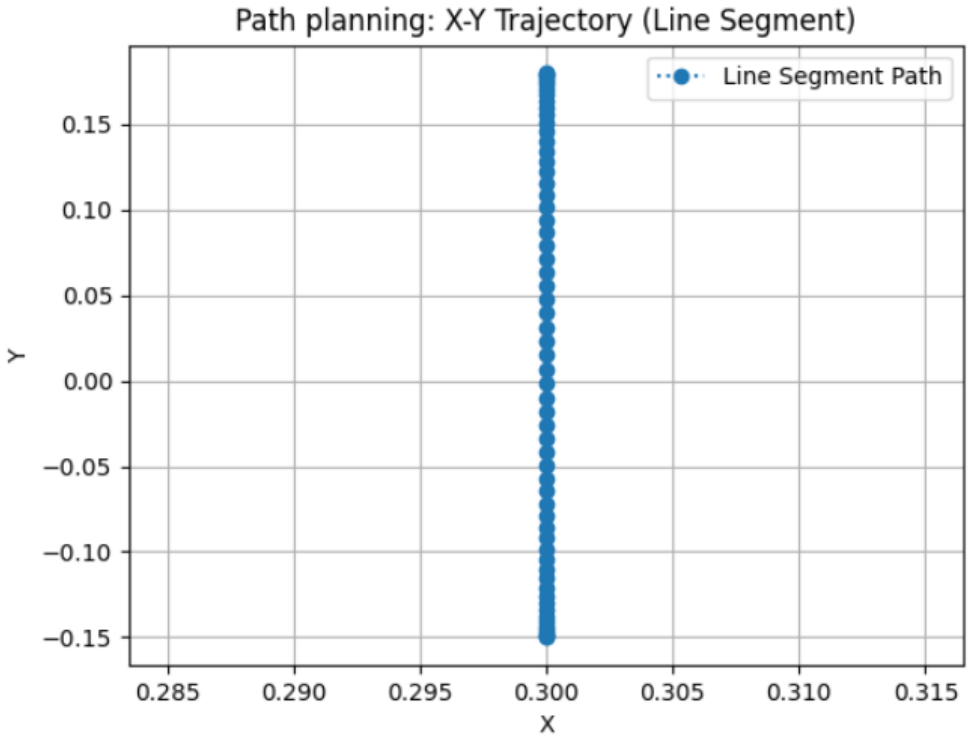
2.



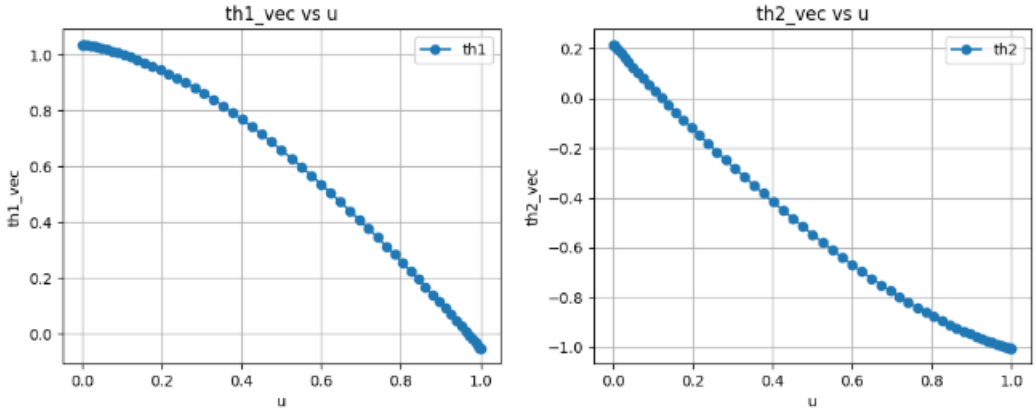
ME381 ROBOTICS

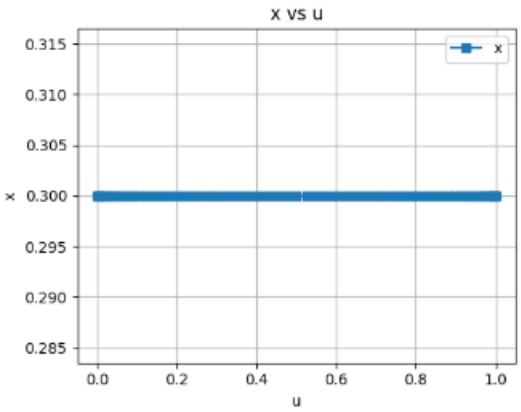
1.

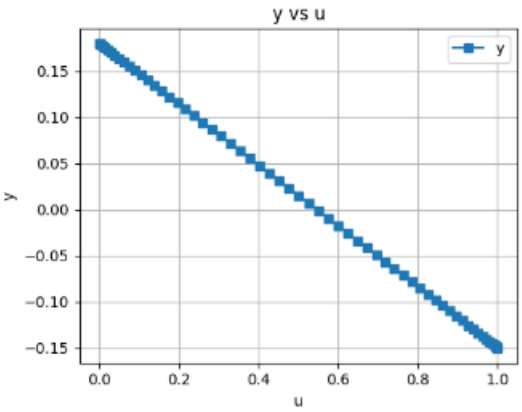


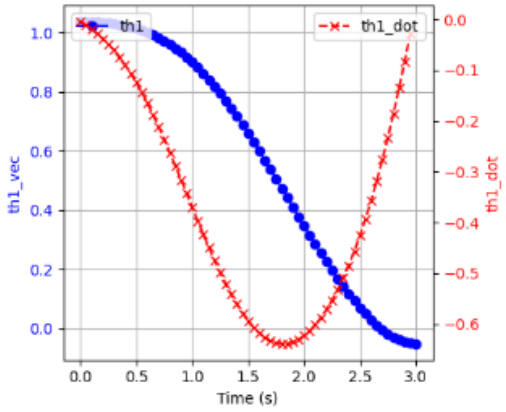


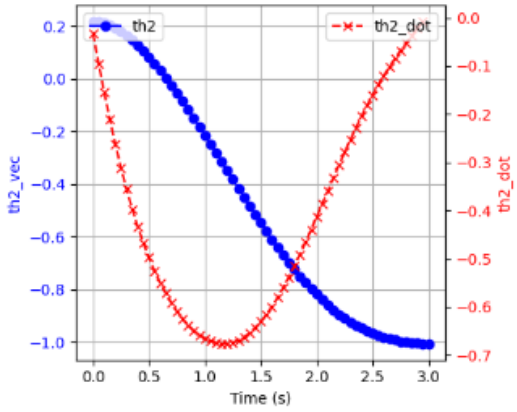
ME381 ROBOTICS

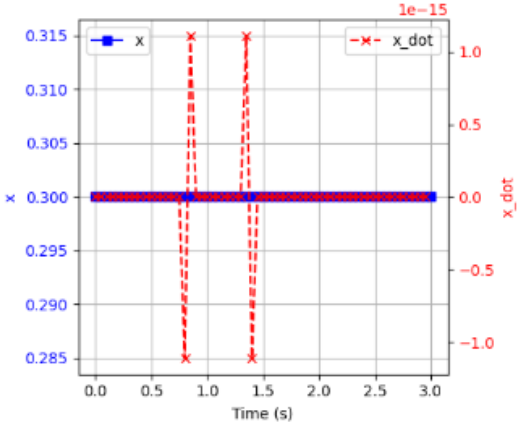


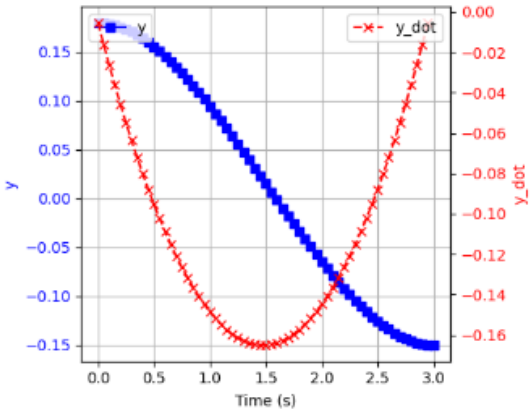




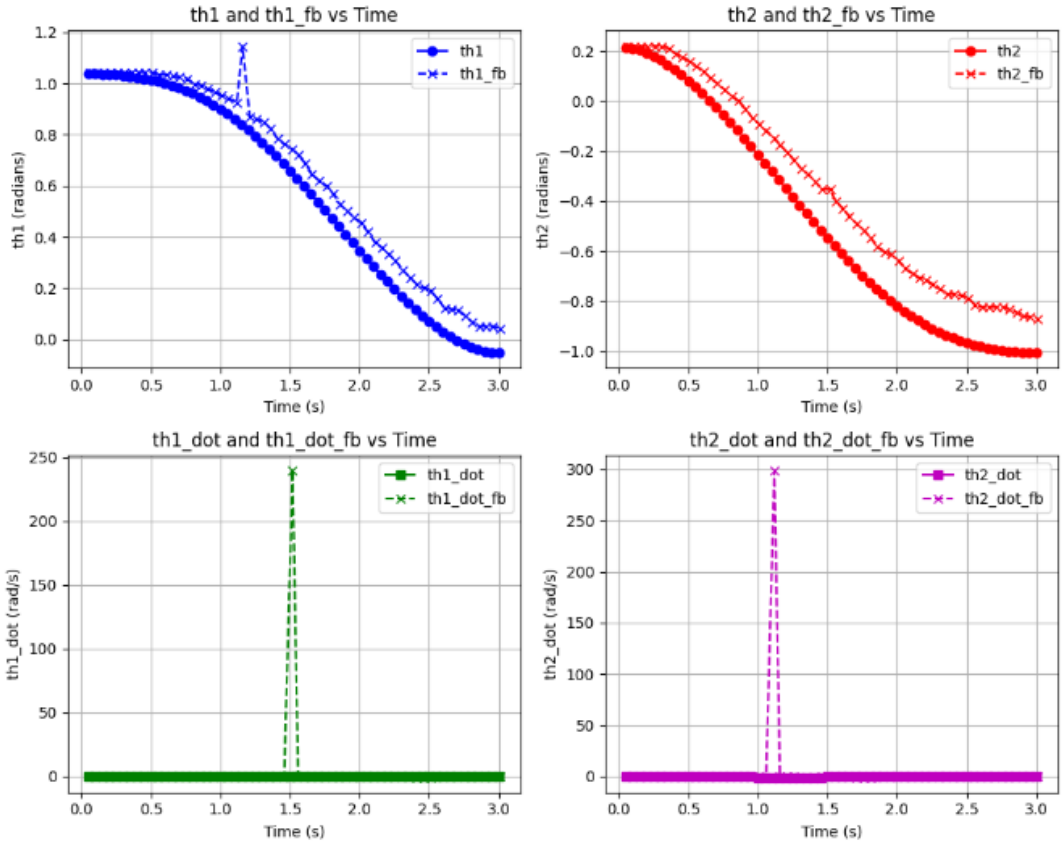






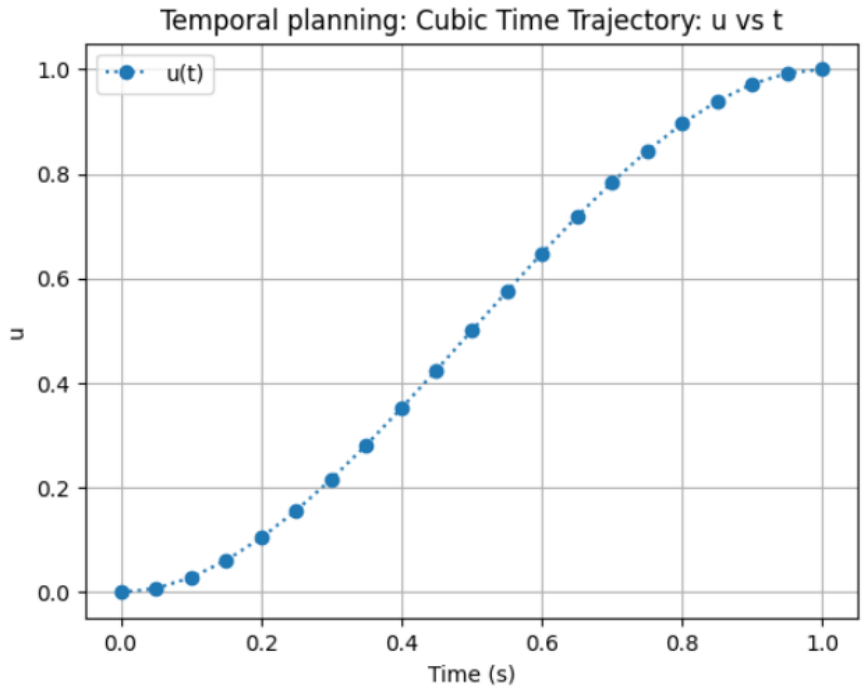


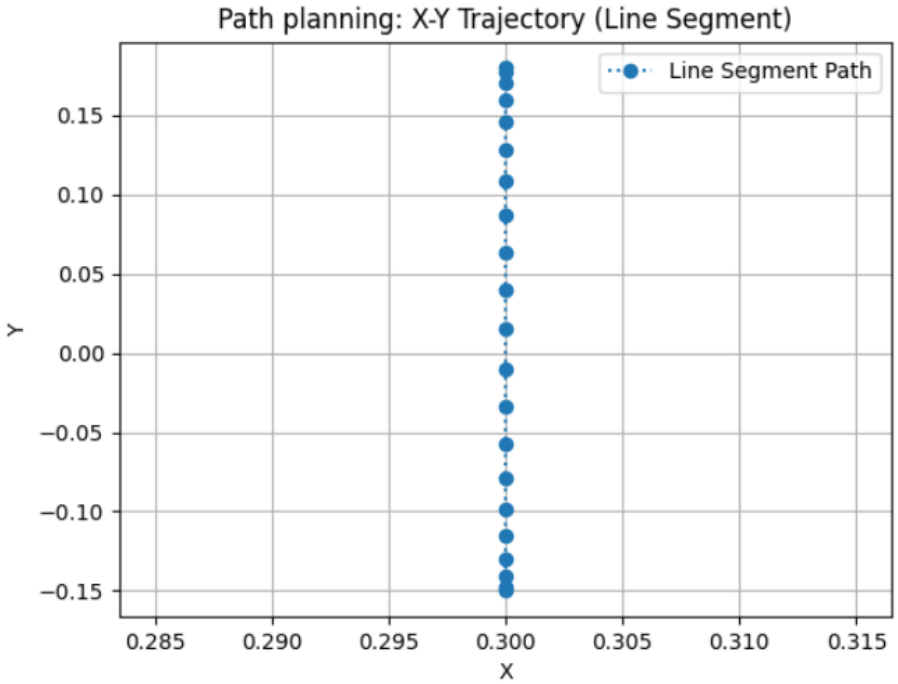
ME381 ROBOTICS



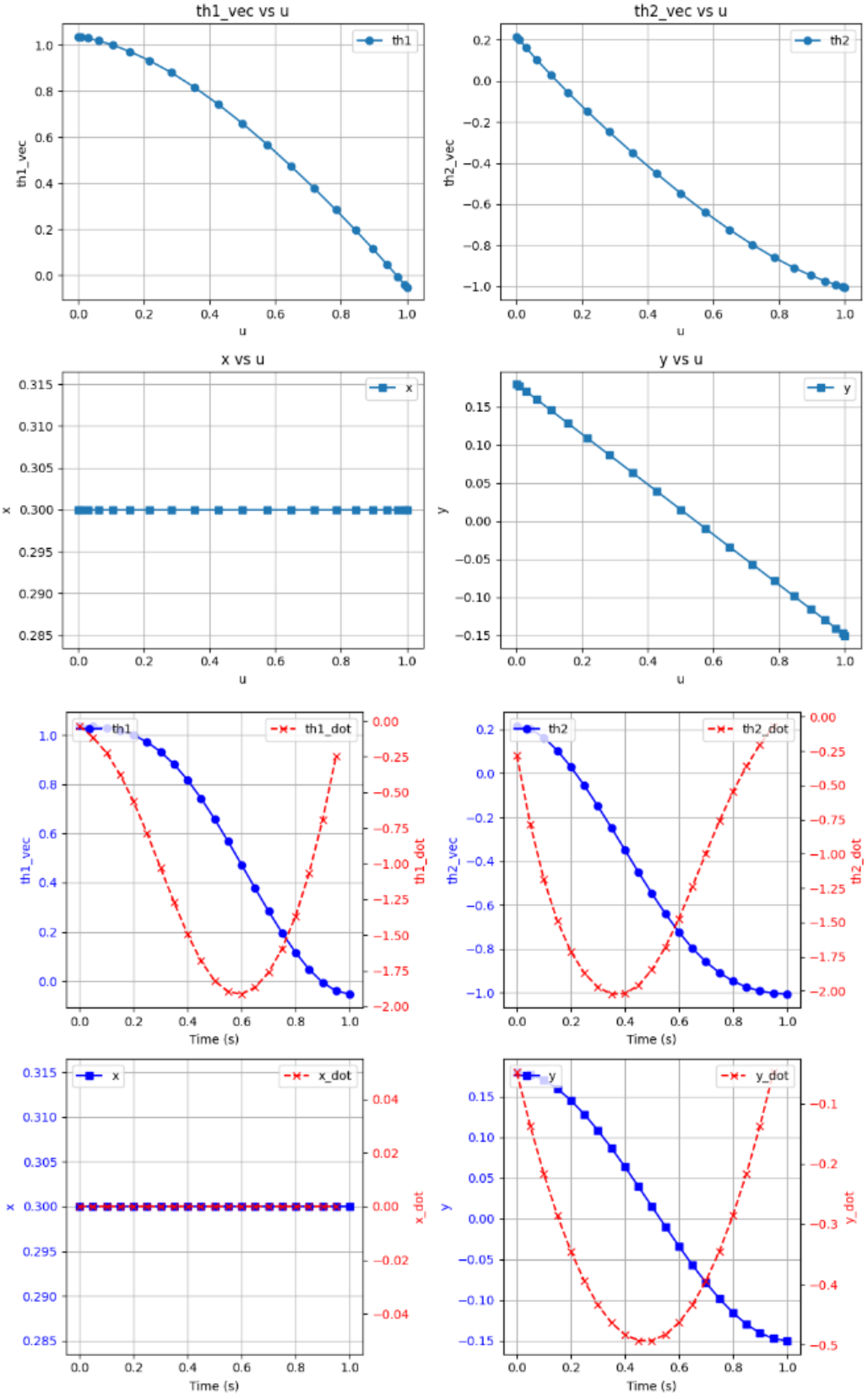
ME381 ROBOTICS

2.





ME381 ROBOTICS



ME381 ROBOTICS